1. (3 Punkte) Die Liste `a = [3, 8, 5, 12, 9, 7, 6, 20, 15]` stellt einen binären Baum dar.
   (a) (1 Punkt) Welche Indizes und Werte haben die Kinder des Knotens am Index 2? Welchen Index und Wert hat der Elternknoten des Knotens am Index 7?
   (b) (1 Punkt) Ist `a` ein Heap? Begründe.
   (c) (1 Punkt) Ist `[2, 5, 4, 6, 3]` ein Heap? Begründe.

Lösung:

a) Kinder von Index 2: Index 5 (Wert 7) und Index 6 (Wert 6). Elternknoten von Index 7: Index $(7-1)//2 = 3$ (Wert 12).

b) Ja, jeder Elternknoten ist kleiner oder gleich seinen Kindern: $3 \leq 8, 5;\ 8 \leq 12, 9;\ 5 \leq 7, 6;\ 12 \leq 20, 15$.

c) Nein: Der Knoten am Index 4 (Wert 3) hat den Elternknoten am Index 1 (Wert 5), und $5 > 3$.

2. (3 Punkte) Schreibe eine Funktion `ist_heap(a)`, die `True` zurückgibt, wenn die Liste `a` ein Heap ist, sonst `False`.

```
'''
>>> ist_heap([3, 8, 5, 12, 9, 7, 6, 20, 15])
True
>>> ist_heap([2, 5, 4, 6, 3])
False
'''
```

Lösung:

```
def ist_heap(a):
    for i in range(1, len(a)):
        if a[(i-1)//2] > a[i]:        # Elternknoten größer als Kind
            return False
    return True
```

3. (2 Punkte) Gegeben ist die Funktion `sift` aus dem Unterricht. Was gibt das folgende Programm aus?

```
a = [9, 2, 4, 5, 3, 8, 6]
sift(a, 0, len(a)-1)
print(a)
```

Lösung: `[2, 3, 4, 5, 9, 8, 6]`: Die 9 versickert, getauscht wird immer mit dem kleineren Kind (erst 2, dann 3).

4. (2 Punkte) Mache aus der Liste einen Heap nach dem Verfahren aus dem Unterricht.

`12 6 3 17 42 5 25 38 9 67 54 1 81`

Lösung: 1 6 3 9 42 5 25 38 17 67 54 12 81

5. (3 Punkte) Mache aus der Liste einen Heap nach dem Verfahren aus dem Unterricht:

`33 12 5 9 39 42 18 7 69 29`

Lösung: 5 7 18 9 29 42 33 12 69 39

6. (4 Punkte) Eine Liste wird mit HeapSort sortiert. Die Zeile zeigt die in einen Heap umgewandelte Liste. Füge für die beiden folgenden Reorganisationen je eine Zeile hinzu.

   `1 6 4 17 28 33 20 92`

   Lösung:

   ```
   4 6 20 17 28 33 92 1
   6 17 20 92 28 33 4 1
   ```

7. (3 Punkte) Eine Liste wird mit HeapSort sortiert. Die Zeile zeigt die in einen Heap umgewandelte Liste. Füge für die beiden folgenden Reorganisationen je eine Zeile hinzu.

   `7 12 11 13 47 42 82 36`

   Lösung:

   ```
   11 12 36 13 47 42 82 7
   12 13 36 82 47 42 11 7
   ```

8. (2 Punkte) Welche Komplexität hat die Laufzeit von HeapSort im best, worst und average-case? In welcher Komplexitätsklasse ist der zusätzliche Platzbedarf? Welche Komplexität hat der Aufbau des Heaps?

   Lösung:

   best, average, worst: $O(n \cdot \log n)$, Platz: $O(1)$, Aufbau des Heaps: $O(n)$

9. (3 Punkte) (a) (2 Punkte) Was gibt das folgende Programm aus?

   ```
   from heapq import heapify, heappush, heappop

   a = [5, 1, 8, 3]
   heapify(a)
   heappush(a, 2)
   print(heappop(a), heappop(a))
   print(a[0])
   ```

   (b) (1 Punkt) Welche Laufzeit haben `heappush` und `heappop` bei einem Heap mit n Elementen?

   Lösung: a) `1 2` und `3`: `heappop` liefert immer das kleinste Element, `a[0]` ist das kleinste der übrigen Elemente 3, 5, 8.

   b) $O(\log n)$